# Beamer+: An Interactive Presentation Tool

Chandra Gummaluru
chandra@cs.toronto.edu
University of Toronto
Toronto, Ontario, Canada

Xiaowu Wu
mika.wu@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Yanting Fan
yanting.fan@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Hasan Malik
hasan.malik@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Chenhao Zhao
chenhao.zhao@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

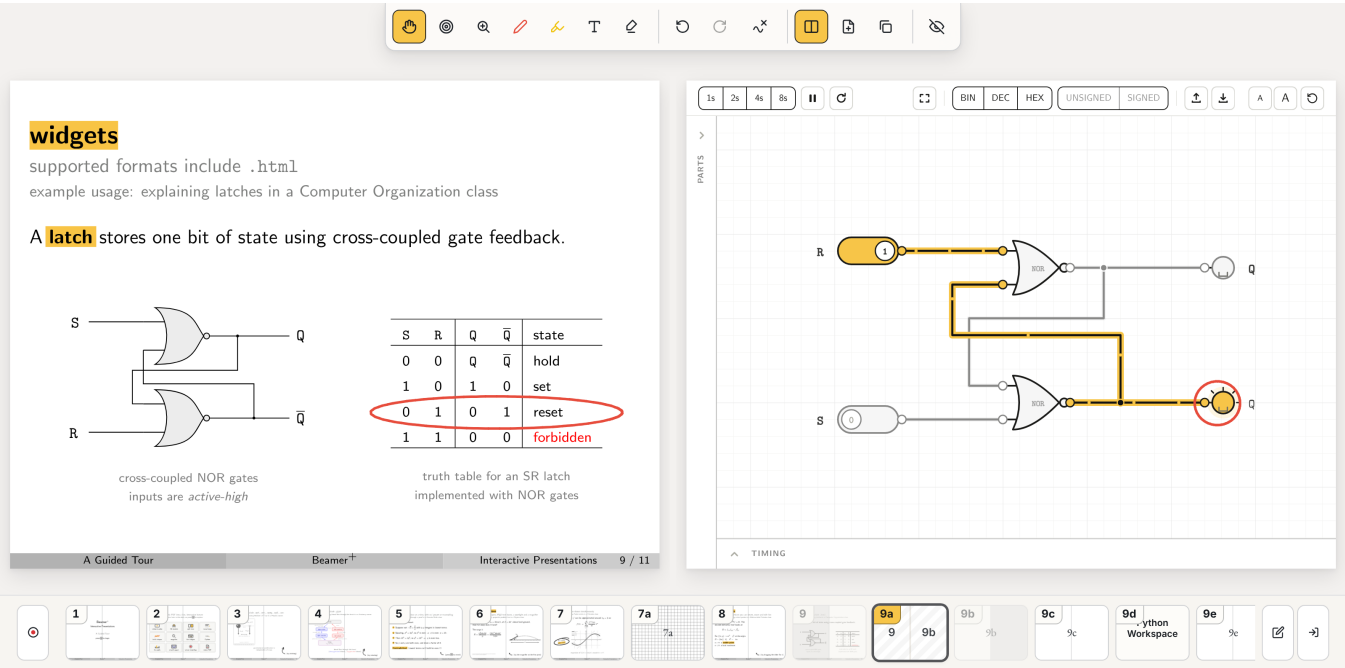



**Figure 1: ~~The~~ Beamer+ ~~user interface~~ during a ~~presentation~~lecture on latches. In split view, a LaTeX Beamer slide (left) sits beside a live circuit-simulator widget (right), with ink drawn over both.**

## Abstract

LaTeX Beamer is widely used for technical presentations, but its static output limits its use in interactive teaching, where an instructor frequently leaves the slides for separate tools such as simulations, coding environments, and polling systems. These switches interrupt the flow of a lecture and cost the attention of everyone in the room. We present Beamer+, a web-based platform that augments existing LaTeX Beamer slide decks with interactive content. A presenter can embed videos, 3D models, and custom interactive widgets directly onto slides, annotate in real time with configurable pens, and use split-view to display two slides at once, all without leaving the presentation. Custom widgets are single self-contained HTML files that can be authored without programming, including with the assistance of large language models, so an instructor can build a tool tailored to their own material. We report on two terms of use across three undergraduate computer science courses, describing how Beamer+ was used to teach topics that are difficult to convey with static slides, such as graph search and digital logic, and reflecting on what worked and what did not. We encountered a limitation in authoring structured content during a lecture, where the interface for reshaping a graph on the fly could not keep pace with a live class. We also observed an unanticipated benefit, as students adopted a demonstration widget on their own devices to generate practice problems we had never assigned. We report these experiences in

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*Conference acronym 'XX, Woodstock, NY*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

enough detail for other instructors to judge where the tool might fit their own teaching.

## CCS Concepts

• **Applied computing** → **Interactive learning environments**; *Computer-assisted instruction*; E-learning; • **Human-centered computing** → *Interactive systems and tools.*

## Keywords

presentation software, interactive learning environments, audience response systems, educational technology, lecture tools, multimedia authoring, CS education, widgets

**ACM Reference Format:**

Chandra Gummaluru, Xiaowu Wu, Yanting Fan, Hasan Malik, and Chenhao Zhao. 2026. Beamer+: An Interactive Presentation Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

LaTeX Beamer has long been the standard tool for creating technical presentations, owing to its philosophy of separating content from design, native support for mathematical and scientific notation, and the ease with which both the source files and output can be shared. However, a significant drawback of the system is that it can only create static presentations, offering limited support for multimedia or interactive elements. While other presentation tools offer some support for multimedia, they lack the qualities that make LaTeX Beamer well suited to technical presentations. They also provide little support for custom interactive elements.

To address these limitations, we developed Beamer+, a web-based platform that augments static slide decks with multimedia and interactive elements. A presenter can embed videos, 3D models, and custom widgets directly into slides, make annotations using configurable pens, and use split-view mode to display two slides simultaneously. The result is a tool that brings the full range of today's interactive content into a presentation, without giving up the benefits of LaTeX Beamer.

Beamer+ grew out of our own experiences within the classroom. In Section 2, we describe the specific challenges that prompted its development. In Section 3, we present the features of Beamer+ that address these challenges. In Section 4, we report on our experience using Beamer+ across several undergraduate computer science courses. Section 5 discusses related work, and Section 6 concludes with a discussion of future directions.

## 2 Motivation

Beamer+ grew out of a recurring problem encountered while teaching an undergraduate algorithms and data structures course. In preparing and presenting a lecture on Dijkstra's algorithm, it became evident that using static slides to cover such a multi-step process came with some drawbacks. First, the slides were laborious to produce. Conveying a single run of the algorithm meant building a separate figure for each step, resulting in an unwieldy set of nearly identical slides. Assembling these figures by hand was time-consuming, and keeping them consistent was error-prone, since a single change to the example had to be propagated carefully across every slide that followed. Second, the sheer number of slides made the example difficult to follow and hard to annotate. Any marking added to emphasize a particular detail would disappear when advancing to the next slide, since each step was a separate static image rather than a persistent canvas. Finally, and most importantly, the presentation was rigid. The aforementioned slide set showed one fixed run of the algorithm on one fixed graph, so when a student asked how the search would change if an edge weight were modified, there was no practical way to explore the question in the moment. Answering it meant either working through the change by hand on the board or deferring it, neither of which preserved the continuity of the lecture.

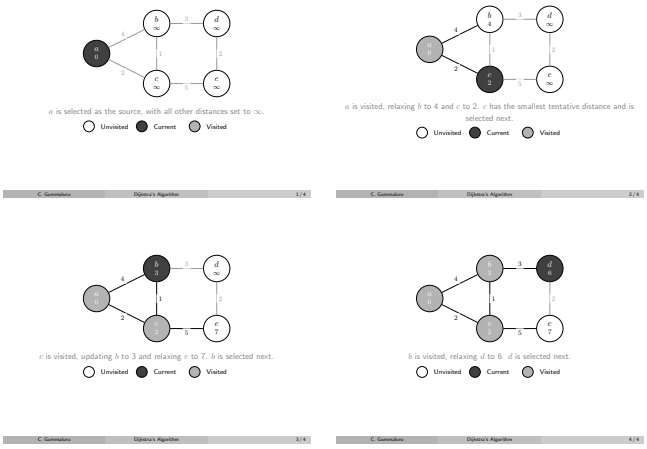



**Figure 2: Four consecutive steps of Dijkstra's algorithm, each differing only in which node is being visited and the tentative distances recorded at each node.**

Each subsequent offering of the course prompted an attempt to do better, but each fell short in an instructive way. Replacing the slides with a pre-rendered animation reduced the size of the deck, but it introduced problems of its own. The animation had to be played in a separate window, and once it was playing there was no natural way to pause and annotate it to discuss a particular step. More importantly, using a pre-rendered animation still meant that there was no easy way to explore students' impromptu questions. Moving the presentation into software with stronger multimedia support made animations easier to embed, but sacrificed the mathematical typesetting and source-based workflow that make LaTeX Beamer well suited to technical material. Rendering Beamer slides as images and importing them into that software restored the typesetting, but made the deck onerous to maintain, since any change meant repeating the entire process.

Across these attempts the underlying problem stayed the same. The interactive content always lived outside the presentation, and uniting the two required either significant manual effort or a separate application. The capability that was missing throughout, the ability to respond to a student's question by exploring it live, is exactly the kind of interactive activity that makes a lecture more

than a static recitation. Such activities are not specific to search algorithms. Many technical concepts are best understood by watching a process unfold, through a simulation, a running program, or an interactive visualization.

Interactive activities of this kind are a well-supported part of effective teaching. In a meta-analysis of 225 studies across science, engineering, and mathematics, Freeman et al. found that active learning improved examination performance and reduced failure rates relative to traditional lecturing [3]. Running a simulation, executing code, polling the room, or working through a problem in a shared workspace are all ways of introducing active learning, but each typically requires its own application, so an instructor must repeatedly switch between the presentation and the tools that support these activities.

Research in cognitive psychology has documented a measurable cost to task switching [10], as attention remains partly fixed on the previous task and lowers performance on the current one [8]. These effects were established with controlled laboratory tasks rather than classroom activities, but an instructor who alternates between explaining material and operating a separate application is engaging in a similar kind of switching, and the resulting loss of attention can diminish the quality of instruction. Students must reorient themselves as well, each time the lecture moves between interfaces. In their review of audience response systems, Kay and LeSage identify the time required to set up and operate such tools as a recurring drawback that can undermine their pedagogical benefit [6], and similar friction can be expected whenever an interactive activity depends on software outside the presentation.

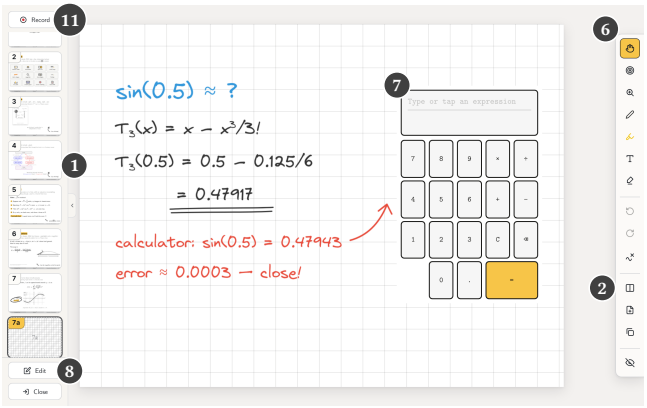
Beamer+ was created to remove this separation. By bringing interactive content directly into LaTeX Beamer presentations, it lets an instructor incorporate simulations, executable code, audience interaction, and other dynamic elements without leaving the slides. The aim is not merely to add multimedia to Beamer, but to make interactive teaching activities a natural part of the presentation itself.

## 3 Features

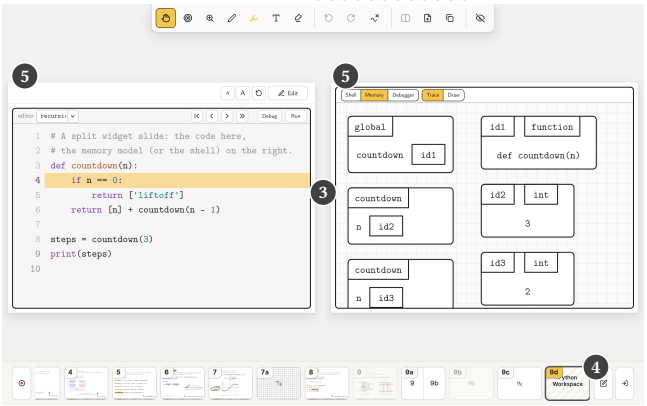
A Beamer+ presentation is a `.zip` file containing a static `.pdf` slide deck along with any interactive overlays and relevant metadata. To create one, a presenter opens their static `.pdf` slide deck and augments it with overlays through the user interface (see Section 3.6). These overlays can be freely positioned and resized on any slide, and since the metadata is stored separately from the `.pdf` file, the original slide deck remains unchanged. A presenter can also insert blank slides to allow space for annotations (see Section 3.3), or create a "split" to automatically display two slides side by side at a designated point in the presentation (see Section 3.2). A presenter can also choose to mark certain slides as hidden to prevent them from being shown during a presentation. When the presentation is ready, it can be saved as a `.zip` file, which can be ~~re-reopened~~ reopened at any time to restore the full presentation.

### 3.1 Navigation

A presenter can navigate between slides using the left and right arrow keys, or jump directly to any slide by clicking its thumbnail in the *slide navigator* (1). Hidden slides are skipped when using the left or right arrow keys, making it easy to include optional or supplementary material without disrupting the flow of a presentation. Slides can also be bookmarked, and bookmarks appear as pins at the top of the navigator so that a key slide is one click away.




(a) ~~Overview~~Single-slide view




(b) ~~Overview (split)~~Split view

**Figure 3: The Beamer+ interface in (a) single-slide view, showing a blank grid page with ink and a calculator widget, and (b) ~~split-view modes~~split view, showing a split widget: a Python workspace whose editor fills the left slide and whose memory model fills the right. Numbered callouts are referenced throughout this section.**

### 3.2 Split-View Mode

A presenter can also display two slides simultaneously using *split-view* (2). The relative size of each pane can be adjusted by dragging the *divider* (3) between them. In split-view mode, ~~thumbnails in the *split slide navigator* are divided into left and right halves, allowing slides to be assigned independently to either pane~~the navigator moves to the bottom of the screen, and a split prepared in advance appears in it as a single thumbnail showing both of its slides (4).

A split can also hold a *split widget* (5), a single widget spanning both slides. The Python workspace, for example, places its editor on the left and a shell, memory model, or debugger on the right, so code run on one slide is traced on the other.

## 3.3 Annotations

A presenter can draw freehand annotations using ~~using the built in~~ the built-in pen, highlighter, and text tools in the *annotation bar* (6), with support for erasing, undoing, or re-doing annotations. Drawing a stroke and holding still snaps it to a clean shape. A spotlight is also available, dimming the slide and highlighting the cursor position to draw audience attention, as is a magnifier that enlarges the region under the cursor. Annotations are saved with the presentation, so they are preserved across sessions.

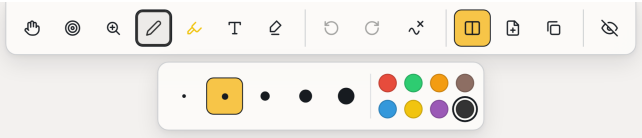


**Figure 4: The annotation bar as it appears in split view, ~~providing configurable pens~~ with the pen selected and ~~highlighters~~its size and colour options open. From left: hand, ~~an eraser~~spotlight, magnifier, pen, highlighter, text, eraser; undo~~and~~, redo, ~~and a spotlight tool~~clear; split view, new page, duplicate; hide slide.**

## 3.4 Overlays

Beamer+ uses the term *overlay* to refer to any interactive or multimedia element layered on top of a slide's static content. A presenter can augment slides with several types of overlay, including videos, audio, 3D models, and interactive widgets (7). The annotation canvas always sits above overlays, so a presenter can write directly on top of any interactive content, just as they would on a static slide.

*3.4.1 Videos and Audio.* Video overlays embed clips (e.g., demonstrations, recorded experiments, or process animations) directly onto a slide from `.mp4` files, and audio overlays do the same for sound clips that need no picture. Standard playback controls are available while presenting, and preset options such as autoplay, looping, and muting let a presenter tailor how a clip behaves when its slide is reached. The clip's playback position can be remembered so a video resumes where it left off when the presenter returns to the slide.

*3.4.2 3D Models.* Model overlays render interactive 3D objects (e.g., molecular geometries, mechanical assemblies, or anatomical structures) on a slide from `.glb` files. The model can be rotated, panned, and zoomed during a presentation to examine detail from any angle. The model's camera viewpoint can be remembered, so a configured angle is retained when the presenter returns to the slide.

*3.4.3 Widgets.* Widgets are the primary mechanism for making static slides dynamic. Beamer+ includes a built-in library of common widgets (e.g., function plotters, calculators, circuit simulators, Python workspaces, coding notebooks, and audience-response tools). A widget can fill the whole slide or, where it supports it, sit in a small box over part of a content slide, as the calculator does in Figure 3a. Built-in widgets use Computer Modern type and TikZ-style outlines, so they read as part of the slide. A widget's state is saved with the presentation, so that it resumes where it left off when the presenter returns to the slide.

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <script id="widget-schema" type="application/json">
    { "fields": [
        { "key": "title", "label": "Title",
          "type": "text", "default": "My Widget" },
        { "key": "count", "label": "Count",
          "type": "number", "default": 5 } ] }
  </script>
</head>
<body>
  <div id="widget-root"></div>
  <script>
    // Called on load and on every property change.
    function init(props) {
      const root = document.getElementById("widget-root");
      root.innerHTML = `<h2>${props.title}</h2>`;
      // Use props.title and/or props.count below...
    }
    init(window.WIDGET_CONFIG || {});
    addEventListener("message", e => {
      if (e.data?.type === "widget-config")
        init(e.data.config);
    });
  </script>
</body>
</html>
```

**Listing 1: Minimal custom widget skeleton. The ~~beamer-schema~~ widget-schema block declares editable properties; the `init` function receives their current values, on load and after every change, and renders the widget content.**

*3.4.4 Custom Widgets.* In addition to the built-in library of widgets, Beamer+ supports custom widgets that allow an instructor or developer to create domain-specific learning tools tailored to their course. A widget is a single self-contained `.html` file. Editable properties are declared in a `JSON` schema block in the file's `<head>`, specifying field labels, types, and defaults. Beamer+ reads this schema and renders the corresponding controls in the ~~editor's Properties~~ widget's settings panel automatically, with no additional configuration required.

A widget can instead declare that it is edited in place, as most built-in widgets do: pressing its *Edit* button turns the widget itself into the form, so that a poll's question and answers are typed where they will appear (Figure 6a). For a presenter without web development experience, modern large language models (LLMs) are well suited to generating custom widgets from a natural-language description. To facilitate this, Beamer+ comes with a `WIDGET_GUIDE.md` file containing the full widget specification, schema syntax, and worked examples, which can be provided directly as context when prompting an LLM.

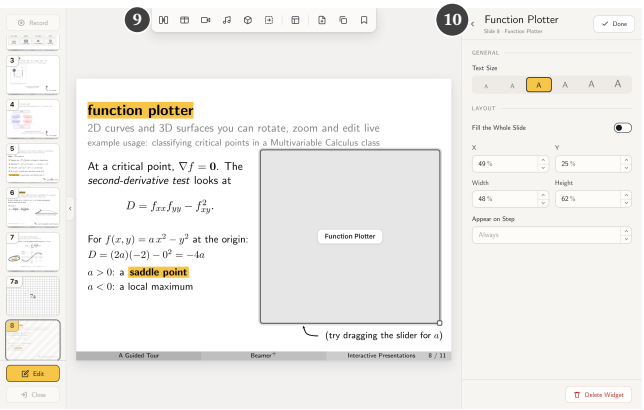
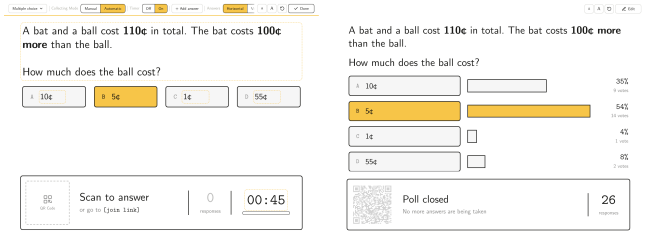



**Figure 5: The Beamer+ editor, with a function-plotter widget selected. Overlays are added from the insert bar and arranged on the slide, and a properties panel exposes the ~~settings~~ layout of the selected overlay.**




**(a) Editing in place** **(b) Results, answer revealed**

**Figure 6: A multiple-choice poll. (a) In edit mode, the question and answers are typed where they appear, the correct answer is marked by clicking it, and the poll's settings sit in its top bar. (b) After the poll closes, the results replace the empty space, and the correct answer is highlighted.**

## 3.5 Reveal Steps

Beamer's \pause uncovers a slide one part at a time, but only by producing a separate page per step, the near-duplicate slides of Section 2. Beamer+ offers the same effect on a single page: a presenter draws numbered *reveal boxes* over a slide, and each press of the right arrow key uncovers the next one before moving on. Overlays can likewise appear on a given step.

## 3.6 Edit Mode

A presentation can be edited by switching into edit mode (8). In this mode, selecting a slide brings it into the main editing area, where overlays and reveal boxes can be added from the *insert bar* (9) and arranged on top of the rendered page. Selecting an overlay exposes a ~~panel containing any relevant settings for the selected overlay~~ *properties panel* (10) containing its size, position, text size, and the step on which it appears; a widget's own settings are edited from the widget itself. Slides can be reordered by dragging the thumbnails in the slide navigator, and blank slides can be inserted at any point as additional working space, supporting the same overlays and annotations as other rendered pages. A presenter can also create a "split," which pairs two existing slides at a ratio chosen ahead of time and displays them side by side when the split is reached while presenting. Slides can also be marked as hidden, excluding them from the presentation while keeping them in the deck for reference or later use. Saving collects the slide deck together with all overlays, annotations, and metadata into a single `.zip` file, which can be reopened in edit mode or used directly to present.

## 3.7 Audience Response

Beamer+ also includes ~~two audience response widgets that~~ an audience response widget whose question types share a common server-side infrastructure~~. Both display~~: multiple choice, true/false, rating scales, numeric estimates, numerical answers, word clouds, fill-in-the-blank, a two-dimensional plot on which each student places a point, and open-ended text. Questions and answers may contain Markdown, code, and LaTeX mathematics, edited in place on the slide (Figure 6a). While a poll is open, the widget displays a QR code that audience members scan on their own devices to submit ~~free-text~~ responses, with each submission broadcast immediately via Socket.IO so the live count updates on the presenter's widget in real time. ~~The open survey widget presents incoming responses as a scrolling live feed and, once the survey is closed, it can dispatch the collected responses~~ A poll closes by hand or when an optional countdown ends, after which its results are drawn on the slide and a correct answer, if one was set, can be revealed (Figure 6b). Open-ended responses can be dispatched to a pluggable AI summarization back-end. Summarizer modules are Python files stored in the ZIP's `ai/` directory, each exporting a single function that groups responses and returns a ranked list of summary and respondent-count pairs. Results are displayed in a navigable carousel. ~~The word cloud widget processes responses client-side: each submission is tokenized, filtered against a stopword list, and accumulated into a frequency map stored in session storage, so the cloud survives slide navigation without re-fetching. The presenter can then switch between the QR collection view and the rendered word cloud at any time.~~

## 3.8 ~~Usage Guide~~

## 3.8 Recording, Notes, and Usage Guide

A lecture can be recorded from within Beamer+ using the *record* button (11), capturing the slides, ink, and widgets with the presenter's microphone. When saving, the presenter can also export a *notes* PDF as a handout, with every visible slide, its annotations, each widget as it was left, and poll results. To support first-time users, Beamer+ provides ~~an integrated usage guide covering presentation loading, navigation, editing, and widget creation, supplemented by fully remappable keyboard shortcut documentation~~a demonstration presentation with a guided tour.

# 4 Classroom Experience

We have used Beamer+ over two terms in three undergraduate computer science courses: an introduction to artificial intelligence, a

course on data structures and algorithms, and a course on computer organization. All three were taught by one of the authors, who used Beamer+ for a handful of lectures in each course, focusing on the topics that are hardest to convey with static slides. We did not conduct a controlled study; instead, this section reports on how the tool was used and what we learned from using it.

The search widget that motivated Beamer+ (Section 2) was used in both the artificial intelligence and the data structures courses, where graph search recurs. Embedded directly on the slide, it animated the progress of an algorithm such as A* and allowed its parameters to be changed during the lecture. When a student asked how the search would behave under a different edge weight, the new value could be entered and the algorithm re-run in front of the class, rather than deferred to office hours or sketched by hand. This was precisely the capability whose absence had originally made the topic awkward to teach.

Using the widget in front of students also revealed a limitation we had underestimated while building it. Editing edge weights on the fly worked well, but changing the structure of the graph itself, adding or removing nodes and edges, was awkward, because the widget had no convenient interface for doing so mid-lecture. A student who asked to see the algorithm run on a differently shaped graph could not always be accommodated in the moment. The tool had been built to escape the rigidity of static slides, yet in this respect it imposed a narrower rigidity of its own. The gap was specifically in the presenter-facing interface for authoring a graph during a lecture, not in the widget's ability to handle different graphs, and addressing it is a clear direction for future work.

That underlying flexibility surfaced in an unexpected way. Although the widget was built as a tool for demonstration at the front of the room, students began using it themselves during and after lecture, generating new graphs on their own devices and running the algorithm on them to practice. A tool intended to support explanation became, with no change on our part, a way for students to create their own practice problems. This was among the more valuable outcomes of the deployment, and one we had neither designed for nor anticipated.

In the computer organization course, Beamer+ was used to teach digital logic, where a circuit and its timing behaviour are difficult to present together on static slides. A logic-simulator widget allowed inputs to be toggled live, split-view paired a circuit diagram with its timing diagram, and the annotation canvas was used to draw the correspondence between a change in the circuit and the resulting edge in the waveform. Across all three courses, ~~the~~ a word-cloud ~~widget~~ poll served as a warm-up that surfaced which concepts students found most confusing, and short video overlays and 3D models were embedded where a static figure was insufficient.

Taken together, our experience suggests that Beamer+ achieved its central aim of keeping a lecture's activities within a single environment: demonstration, comparison, annotation, and audience response were carried out without leaving the presentation. The main shortcoming we encountered was the difficulty of authoring structured content on the fly, and the main surprise was the tool's adoption by students for their own practice. Both point toward the same next step: a faster interface for creating and editing interactive content, usable by instructors and students alike.

## 5 Related Work

Slide-based software such as PowerPoint, Keynote, Google Slides, and LaTeX Beamer arranges content as a sequence of slides, and web-based tools such as reveal.js bring the same model into the browser. A body of research has extended this model in several directions. Zoomable user interfaces, popularized by Prezi and investigated in research systems such as CounterPoint [4] and Fly [9], replace the fixed sequence with a spatial canvas, and pen-based systems such as Classroom Presenter bring handwriting into the lecture, allowing an instructor to ink directly on slides and share digital ink with students [1]. Closest to our work is MindXpres, a web-based, extensible presentation platform whose plug-in architecture allows interaction components to be added to a presentation rather than chosen from a fixed set [11]. Beamer+ differs in two respects that bear on classroom adoption: it augments an existing `.pdf` deck in place rather than requiring content to be re-authored in a dedicated format, and its custom widgets can be created without programming, including from a natural-language description.

The activities an instructor reaches for outside the deck have each been studied as a medium in their own right. Interactive simulations let students manipulate a model and observe its behaviour directly, as in the PhET project, which reports learning gains when its simulations replace static presentation of the same material [13]. Explorable explanations embed reactive content that lets a reader change an author's assumptions and see the consequences [12], and authoring tools such as Idyll reduce the effort of building such interactive articles [2]. Computational notebooks place executable code beside prose and output, descending from Knuth's notion of literate programming [7], and have been used in the classroom to run and edit code live rather than display a static listing [5]. Audience response systems such as Kahoot, Mentimeter, and Poll Everywhere bring student interaction into the lecture, but offer a fixed menu of interaction types. Each of these media is typically delivered by a separate application; Beamer+ brings them into a single presentation surface and additionally allows arbitrary custom interactive content to be embedded alongside them.

## 6 Conclusion

We presented Beamer+, a web-based tool that augments LaTeX Beamer slide decks with interactive overlays, and reported on two terms of use across three undergraduate computer science courses. The design rests on a single premise, that the cost of leaving the slides for a separate application is paid not only in lost time but in the attention of everyone in the room. By bringing simulations, executable code, audience response, and other dynamic content onto the slide itself, Beamer+ allowed topics that are hard to convey with static slides to be taught within one environment, combining demonstration, comparison, annotation, and audience response without leaving the presentation. Custom widgets extend this beyond a fixed menu of interactions, since an instructor can author a tool for their own material as a single HTML file with no web development experience required.

Putting the tool in front of students taught us more than building it. Our experience surfaced a limitation, the difficulty of authoring structured content such as a graph during a lecture, where the demonstration interface could not keep pace with a class that

wanted to reshape an example on the spot. It also surfaced an unanticipated benefit, the adoption of an instructor's demonstration widget by students, who used it on their own devices to generate practice problems we had never assigned. A tool built for the front of the room found a second use in the hands of the people it was meant to reach.

These observations shape our future work. The most pressing improvement is a faster interface for creating and editing structured interactive content on the fly, which would address the limitation we encountered and support the student-driven practice we observed. We also plan to broaden deployment beyond a single instructor and to study more systematically how the tool affects teaching and learning. By describing both what worked and what did not in concrete terms, we hope to give other instructors a clear sense of where Beamer+ might serve their own classrooms. Beamer+ and its widget specification are available at https://example.com/beamerplus.

## Acknowledgments

Removed for review.

## References

[1] Richard Anderson, Ruth Anderson, Peter Davis, Natalie Linnell, Craig Prince, Valentin Razmov, and Fred Videon. 2007. Classroom Presenter: Enhancing Interactive Education with Digital Ink. *Computer* 40, 9 (2007), 56–61.

[2] Matthew Conlen and Jeffrey Heer. 2018. Idyll: A Markup Language for Authoring and Publishing Interactive Articles on the Web. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST '18)*. ACM, 977–989.

[3] Scott Freeman, Sarah L. Eddy, Miles McDonough, Michelle K. Smith, Nnadozie Okoroafor, Hannah Jordt, and Mary Pat Wenderoth. 2014. Active Learning Increases Student Performance in Science, Engineering, and Mathematics. *Proceedings of the National Academy of Sciences* 111, 23 (2014), 8410–8415. doi:10.1073/pnas.1319030111

[4] Lance Good and Benjamin B. Bederson. 2002. Zoomable User Interfaces as a Medium for Slide Show Presentations. *Information Visualization* 1, 1 (2002), 35–49.

[5] Jeremiah W. Johnson. 2020. Benefits and Pitfalls of Jupyter Notebooks in the Classroom. In *Proceedings of the 21st Annual Conference on Information Technology Education (SIGITE '20)*. ACM, 32–37.

[6] Robin H. Kay and Ann LeSage. 2009. Examining the Benefits and Challenges of Using Audience Response Systems: A Review of the Literature. *Computers & Education* 53, 3 (2009), 819–827. doi:10.1016/j.compedu.2009.05.001

[7] Donald E. Knuth. 1984. Literate Programming. *Comput. J.* 27, 2 (1984), 97–111.

[8] Sophie Leroy. 2009. Why Is It So Hard to Do My Work? The Challenge of Attention Residue When Switching Between Work Tasks. *Organizational Behavior and Human Decision Processes* 109, 2 (2009), 168–181. doi:10.1016/j.obhdp.2009.04.002

[9] Leonhard Lichtschlag, Thorsten Karrer, and Jan Borchers. 2009. Fly: A Tool to Author Planar Presentations. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. ACM, 547–556.

[10] Stephen Monsell. 2003. Task Switching. *Trends in Cognitive Sciences* 7, 3 (2003), 134–140. doi:10.1016/S1364-6613(03)00028-7

[11] Reinout Roels and Beat Signer. 2014. MindXpres: An Extensible Content-Driven Cross-Media Presentation Platform. In *Web Information Systems Engineering – WISE 2014 (Lecture Notes in Computer Science, Vol. 8787)*. Springer, 215–230.

[12] Bret Victor. 2011. Explorable Explanations. http://worrydream.com/ExplorableExplanations/.

[13] Carl E. Wieman, Wendy K. Adams, and Katherine K. Perkins. 2008. PhET: Simulations That Enhance Learning. *Science* 322, 5902 (2008), 682–683.